

On the EUF-CMA Security of TL-DSA: A Formal Reduction to Module-LWE over Ternary Polynomial Rings

Applied Physics Division
Capomastro Holdings Ltd.
research@capomastro.com

March 2026

Abstract

We present a formal security analysis of TL-DSA (Ternary Lattice Digital Signature Algorithm), a Fiat–Shamir with Aborts signature scheme operating in the polynomial ring $R_3 = \mathbb{Z}_3[X]/(X^{256} + 1)$ with balanced ternary coefficients $\{-1, 0, +1\}$. TL-DSA is structurally analogous to ML-DSA (FIPS 204/Dilithium) but specialized to the ternary domain. We prove that TL-DSA achieves existential unforgeability under chosen-message attack (EUF-CMA) in the random oracle model, assuming the hardness of the Module Short Integer Solution (MSIS) problem over R_3 . We further show that MSIS over R_3 is at least as hard as Module-LWE over R_3 via the standard SIS–LWE duality. A structural property unique to the ternary setting—that rejection sampling becomes trivially satisfied when $q = 3$ —is identified and analyzed, yielding deterministic (non-aborting) signing with perfect zero-knowledge simulation. Concrete security bounds are derived for all three parameter sets (TL-DSA-44, TL-DSA-65, TL-DSA-87) targeting 128, 192, and 256 bits of classical security respectively.

Keywords: Post-quantum signatures, lattice-based cryptography, Module-LWE, Module-SIS, Fiat–Shamir, ternary arithmetic, balanced ternary.

Contents

1	Introduction	2
2	Preliminaries	2
2.1	Notation	2
2.2	The Ternary Polynomial Ring	3
2.3	Module-SIS and Module-LWE over R_3	3
2.4	Centered Binomial Distribution over $\mathbb{Z}_3$	3
2.5	Ternary Sponge Hash Function	3
3	TL-DSA Construction	4
3.1	Parameters	4
3.2	Key Generation	4
3.3	Signing	5
3.4	Verification	5
4	The Ternary Non-Abort Property	5

5	EUFCMA Security Proof	6
5.1	Security Model	6
5.2	Main Theorem	6
5.3	Proof of Theorem 5.2	7
6	Module-SIS to Module-LWE Duality	8
7	Concrete Security Estimates	9
7.1	Lattice Dimension and Hardness	9
7.2	Hash Function Security	9
7.3	Sizes	9
8	Discussion	10
8.1	Comparison to ML-DSA	10
8.2	Security Considerations Specific to $q = 3$	10
8.3	Open Problems	10
9	Conclusion	11
A	Implementation Correspondence	11
B	Test Vector Correspondence	11

1 Introduction

The standardization of lattice-based cryptographic schemes by NIST—ML-KEM (FIPS 203) and ML-DSA (FIPS 204)—has established Module-LWE as the foundational hardness assumption for post-quantum public-key cryptography. Both standards operate over polynomial rings $\mathbb{Z}_q[X]/(X^n + 1)$ with large moduli ($q = 3329$ for ML-KEM, $q = 8380417$ for ML-DSA), chosen primarily to enable efficient Number Theoretic Transform (NTT) multiplication.

TL-DSA takes a fundamentally different approach: it operates in the minimal nontrivial polynomial ring $R_3 = \mathbb{Z}_3[X]/(X^{256} + 1)$, where the modulus $q = 3$ coincides with the coefficient domain of balanced ternary $\{-1, 0, +1\}$. This specialization is motivated by the PlenumNET ternary computing platform, where all arithmetic naturally operates in $\text{GF}(3)$.

This paper provides three contributions:

1. **Formal specification** of TL-DSA extracted from the production Rust implementation (Section 3), ensuring the analyzed scheme matches the deployed code.
2. **EUFCMA security proof** in the random oracle model (Section 5), reducing forgery to solving Module-SIS over R_3 , with an explicit reduction to Module-LWE via the standard duality (Section 6).
3. **Identification of the ternary non-abort property** (Section 4): when $q = 3$, the masking vector $\mathbf{y}$ perfectly hides the secret through modular addition, eliminating the rejection sampling that dominates signing cost in standard Dilithium.

2 Preliminaries

2.1 Notation

Bold lowercase letters ($\mathbf{s}, \mathbf{y}, \mathbf{z}$) denote vectors over R_3 . Bold uppercase letters ($\mathbf{A}$) denote matrices over R_3 . Regular lowercase (c, f) denote individual ring elements. We write $\|\mathbf{v}\|_\infty$ for

the maximum absolute value of any coefficient across all polynomials in $\mathbf{v}$. For a polynomial $f \in R_3$, $\text{wt}(f)$ denotes the Hamming weight (number of nonzero coefficients).

2.2 The Ternary Polynomial Ring

Definition 2.1 (Ternary Polynomial Ring). Let $n = 256$. The ternary polynomial ring is

$$R_3 = \mathbb{Z}_3[X]/(X^n + 1),$$

where $\mathbb{Z}_3 = \{-1, 0, +1\}$ denotes the integers modulo 3 in balanced representation. Ring addition is componentwise modulo 3. Ring multiplication is polynomial multiplication reduced modulo $X^n + 1$, with coefficient reduction modulo 3 via the map $r \mapsto ((r \bmod 3) + 3) \bmod 3$ followed by $\{0 \mapsto 0, 1 \mapsto 1, 2 \mapsto -1\}$.

Remark 2.2 (Implementation correspondence). This ring is implemented in `ternary_lattice.rs` (lines 58–214). The `mod3` function (line 67) performs the balanced reduction. Ring multiplication (`ring_mul`, line 185) uses schoolbook convolution with the $X^n + 1$ negacyclic reduction: for indices $i + j \geq n$, the product term is subtracted rather than added.

2.3 Module-SIS and Module-LWE over R_3

Definition 2.3 (Module-SIS over R_3). For parameters $k, l \in \mathbb{Z}^+$ and bound $\beta \in \mathbb{Z}^+$, the $\text{MSIS}_{k,l,\beta}$ problem over R_3 is: given a uniformly random matrix $\mathbf{A} \in R_3^{k \times l}$, find a nonzero vector $\mathbf{x} \in R_3^l$ such that $\mathbf{A} \cdot \mathbf{x} = \mathbf{0} \in R_3^k$ and $\|\mathbf{x}\|_\infty \leq \beta$.

Definition 2.4 (Module-LWE over R_3). For parameters $k \in \mathbb{Z}^+$ and a noise distribution χ over R_3 , the $\text{MLWE}_{k,\chi}$ problem over R_3 is: given $(\mathbf{A}, \mathbf{b})$ where $\mathbf{A} \xleftarrow{\$} R_3^{k \times k}$ and either $\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$ for $\mathbf{s}, \mathbf{e} \leftarrow \chi^k$ or $\mathbf{b} \xleftarrow{\$} R_3^k$, distinguish which case holds.

Remark 2.5 (Ternary specialization). When $q = 3$, every element of R_3 has $\|\cdot\|_\infty \leq 1$. Both the uniform distribution and any noise distribution over R_3 produce coefficients in $\{-1, 0, +1\}$. This means Module-LWE and Module-SIS over R_3 are problems about the algebraic structure of R_3 rather than about distinguishing small errors from uniform—the “errors” are indistinguishable from uniform elements because they share the same support.

2.4 Centered Binomial Distribution over $\mathbb{Z}_3$

Definition 2.6 (CBD_η over $\mathbb{Z}_3$). For parameter $\eta \in \mathbb{Z}^+$, sample $a_1, \dots, a_\eta, b_1, \dots, b_\eta$ uniformly from $\{0, 1, 2\}$ (equivalently, from the output of the ternary sponge). Output $(\sum_{i=1}^\eta a_i - \sum_{i=1}^\eta b_i) \bmod 3$, interpreted in balanced form.

This is implemented in `sample_cbd_ternary` (line 423 of `ternary_lattice.rs`) with $\eta = 2$ for all TL-DSA variants.

2.5 Ternary Sponge Hash Function

TL-DSA uses a custom ternary sponge construction $H : \{-1, 0, +1\}^* \rightarrow \{-1, 0, +1\}^m$ as its hash function, implemented in `sponge.rs`. The sponge has state size 729 trits ($= 3^6$), rate 243 trits ($= 3^5$), capacity 486 trits, and performs 27 rounds of a nonlinear permutation per absorption.

In the security proof, we model H as a random oracle $\mathcal{H} : \{-1, 0, +1\}^* \rightarrow \{-1, 0, +1\}^{243}$.

Table 1: TL-DSA parameter sets, extracted from `tl_dsa.rs` lines 73–103.

Variant	n	k	l	η	γ	τ	Security (bits)	NIST Level
TL-DSA-44	256	4	4	2	1	39	128	2
TL-DSA-65	256	6	5	2	1	49	192	3
TL-DSA-87	256	8	7	2	1	60	256	5

3 TL-DSA Construction

3.1 Parameters

The parameters are:

- $n = 256$: polynomial degree (ring dimension).
- k : number of rows in the public matrix $\mathbf{A}$ and dimension of $\mathbf{t}$.
- l : number of columns in $\mathbf{A}$ and dimension of the secret $\mathbf{s}_1$.
- $\eta = 2$: CBD parameter for secret key sampling.
- $\gamma = 1$: ℓ_∞ rejection bound for signatures.
- τ : Hamming weight of challenge polynomial c .

3.2 Key Generation

```

 $\rho \leftarrow \mathcal{H}(\text{seed}||0)$  // 243 trits, matrix seed
 $\sigma \leftarrow \mathcal{H}(\text{seed}||1)$  // 243 trits, noise seed
 $\xi \leftarrow \mathcal{H}(\text{seed}||0||1||-1)$  // 243 trits, signing seed
 $\mathbf{A} \leftarrow \text{ExpandA}(\rho) \in R_3^{k \times k}$  // uniform ternary matrix
 $\mathbf{s}_1 \leftarrow \text{CBD}_\eta(\sigma) \in R_3^l$  // short secret vector
 $\mathbf{t} \leftarrow \mathbf{A} \cdot \mathbf{s}_1 \in R_3^k$  // public key vector
return  $(pk = (\rho, \mathbf{t}), sk = (\rho, \mathbf{s}_1, \mathbf{t}, \xi))$ 

```

Algorithm 1: KeyGen(seed)

Remark 3.1 (Zero error term). Unlike ML-DSA, TL-DSA computes $\mathbf{t} = \mathbf{A} \cdot \mathbf{s}_1$ without an additive error term $\mathbf{s}_2$. In the implementation (`tl_dsa.rs` line 211), $\mathbf{s}_2$ is initialized as the zero vector. This is justified by the ternary non-abort property (Section 4): since $q = 3$, the distribution $\mathbf{A} \cdot \mathbf{s}_1$ is already pseudorandom under MSIS hardness, and adding noise would not improve security because the noise distribution equals the uniform distribution over R_3 .

3.3 Signing

Input: Secret key $sk = (\rho, \mathbf{s}_1, \mathbf{t}, \xi)$, message $\mu \in \{-1, 0, +1\}^*$

```

A  $\leftarrow$  ExpandA( $\rho$ )
 $\bar{\mu} \leftarrow \mathcal{H}(\text{trits}(\mathbf{t}) \parallel \mu)$  // message representative
for  $\kappa = 0, 1, 2, \dots$  do
    y  $\leftarrow$  ExpandMask( $\xi, \bar{\mu}, \kappa$ )  $\in R_3^l$  // ternary masking vector
    w  $\leftarrow$  A  $\cdot$  y  $\in R_3^k$ 
     $\tilde{c} \leftarrow \mathcal{H}(\bar{\mu} \parallel \text{trits}(\mathbf{w}))$  // challenge hash
    c  $\leftarrow$  SampleChallenge( $\tilde{c}$ )  $\in R_3$  // weight- $\tau$  ternary poly
    z  $\leftarrow$  y + c  $\cdot$  s1  $\in R_3^l$ 
    if  $\|\mathbf{z}\|_\infty \leq \gamma$  then
        return  $\sigma = (\mathbf{z}, \tilde{c})$ 
    end
end

```

Algorithm 2: Sign(sk, μ)

Remark 3.2 (Challenge polynomial). SampleChallenge($\tilde{c}$) produces a polynomial $c \in R_3$ with exactly τ nonzero coefficients, each in $\{-1, +1\}$. This is implemented in `sample_challenge` (line 157 of `tl_dsa.rs`).

3.4 Verification

Input: Public key $pk = (\rho, \mathbf{t})$, message μ , signature $\sigma = (\mathbf{z}, \tilde{c})$

```

if  $\|\mathbf{z}\|_\infty > \gamma$  then return  $\perp$ 
A  $\leftarrow$  ExpandA( $\rho$ )
c  $\leftarrow$  SampleChallenge( $\tilde{c}$ )
 $\mathbf{w}' \leftarrow$  A  $\cdot$  z - c  $\cdot$  t  $\in R_3^k$ 
 $\bar{\mu} \leftarrow \mathcal{H}(\text{trits}(\mathbf{t}) \parallel \mu)$ 
 $\tilde{c}' \leftarrow \mathcal{H}(\bar{\mu} \parallel \text{trits}(\mathbf{w}'))$ 
return [ $\tilde{c}' = \tilde{c}$ ]

```

Algorithm 3: Verify(pk, μ, σ)

Proposition 3.3 (Correctness). *For any valid key pair $(pk, sk) \leftarrow \text{KeyGen}$ and message μ , if $\sigma \leftarrow \text{Sign}(sk, \mu)$ succeeds, then $\text{Verify}(pk, \mu, \sigma) = 1$.*

Proof. Let $\sigma = (\mathbf{z}, \tilde{c})$ where $\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}_1$. In verification:

$$\begin{aligned}
 \mathbf{w}' &= \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t} \\
 &= \mathbf{A}(\mathbf{y} + c \cdot \mathbf{s}_1) - c \cdot \mathbf{A} \cdot \mathbf{s}_1 \\
 &= \mathbf{A} \cdot \mathbf{y} + c \cdot \mathbf{A} \cdot \mathbf{s}_1 - c \cdot \mathbf{A} \cdot \mathbf{s}_1 \\
 &= \mathbf{A} \cdot \mathbf{y} = \mathbf{w}.
 \end{aligned}$$

Since $\mathbf{w}' = \mathbf{w}$, the recomputed challenge hash $\tilde{c}' = \mathcal{H}(\bar{\mu} \parallel \text{trits}(\mathbf{w})) = \tilde{c}$. □

4 The Ternary Non-Abort Property

We identify a structural property unique to TL-DSA's ternary instantiation that has significant implications for both security and efficiency.

Theorem 4.1 (Ternary Non-Abort). *When $q = 3$ and $\gamma = 1$, the rejection condition $\|\mathbf{z}\|_\infty > \gamma$ in Algorithm 2 is never satisfied. Every signing attempt succeeds on the first iteration ($\kappa = 0$).*

Proof. Since all arithmetic is in R_3 , every polynomial in R_3 has coefficients in $\{-1, 0, +1\}$. Therefore $\|f\|_\infty \leq 1$ for all $f \in R_3$. In particular:

- $\mathbf{y} \in R_3^l$ implies $\|\mathbf{y}\|_\infty \leq 1$.
- $c \in R_3$ and $\mathbf{s}_1 \in R_3^l$ implies $c \cdot \mathbf{s}_{1,i} \in R_3$ for each component, so $\|c \cdot \mathbf{s}_1\|_\infty \leq 1$.
- $\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}_1 \in R_3^l$ implies $\|\mathbf{z}\|_\infty \leq 1 = \gamma$.

The rejection condition $\|\mathbf{z}\|_\infty > 1$ is never satisfied because R_3 is closed under addition and multiplication with $\|\cdot\|_\infty \leq 1$. $\square$

Corollary 4.2 (Deterministic Signing). *TL-DSA signing is deterministic: given (sk, μ) , the signature σ is uniquely determined. The for-loop in Algorithm 2 always terminates at $\kappa = 0$.*

Theorem 4.3 (Perfect Zero-Knowledge Simulation). *The signature $\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}_1 \in R_3^l$ is uniformly distributed over R_3^l conditioned on $\mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t} = \mathbf{w}$, regardless of the secret key $\mathbf{s}_1$.*

Proof. The masking vector $\mathbf{y}$ is derived from the signing seed ξ , the message representative $\bar{\mu}$, and the attempt counter $\kappa = 0$ via the random oracle $\mathcal{H}$. In the random oracle model, $\mathbf{y}$ is uniformly distributed over R_3^l .

Since $(\mathbb{Z}_3, +)$ is a group, for any fixed $c \cdot \mathbf{s}_1 \in R_3^l$, the map $\mathbf{y} \mapsto \mathbf{y} + c \cdot \mathbf{s}_1$ is a bijection on R_3^l . Therefore $\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}_1$ is uniformly distributed over R_3^l whenever $\mathbf{y}$ is, independent of $\mathbf{s}_1$.

This means a simulator without knowledge of $\mathbf{s}_1$ can produce identically distributed transcripts: sample $\mathbf{z} \xleftarrow{\$} R_3^l$, compute $\mathbf{w} = \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}$, and program the random oracle accordingly. $\square$

Remark 4.4 (Contrast with ML-DSA). In ML-DSA (Dilithium), $q = 8380417$ and $\gamma_1 \gg \|c \cdot \mathbf{s}_1\|_\infty$. The masking vector $\mathbf{y}$ is sampled from $[-\gamma_1, \gamma_1]$, and rejection sampling ensures that $\mathbf{z}$ is statistically close to uniform on $[-\gamma_1 + \beta, \gamma_1 - \beta]$, which leaks no information about $\mathbf{s}_1$. The expected number of signing attempts is $\approx e^{l \cdot n \cdot 2\beta / (2\gamma_1 + 1)}$, typically 4–7 iterations for ML-DSA-65.

In TL-DSA, the group structure of $\mathbb{Z}_3$ provides *perfect* masking for free: the one-time pad property of modular addition eliminates any need for rejection sampling or statistical closeness arguments. This reduces signing to exactly one attempt with zero information leakage.

5 EUF-CMA Security Proof

5.1 Security Model

Definition 5.1 (EUF-CMA Security). A signature scheme $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ is (ϵ, q_S, q_H) -EUF-CMA secure if for all adversaries $\mathcal{A}$ making at most q_S signing queries and q_H random oracle queries:

$$\text{Adv}_{\Pi}^{\text{EUF-CMA}}(\mathcal{A}) = \Pr \left[\text{Verify}(pk, \mu^*, \sigma^*) = 1 \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen} \\ (\mu^*, \sigma^*) \leftarrow \mathcal{A}^{\text{Sign}(sk, \cdot), \mathcal{H}(\cdot)}(pk) \end{array} \right] \leq \epsilon,$$

where μ^* was not queried to the signing oracle.

5.2 Main Theorem

Theorem 5.2 (EUF-CMA Security of TL-DSA). *In the random oracle model, for any EUF-CMA adversary $\mathcal{A}$ against TL-DSA making at most q_S signing queries and q_H random oracle queries, there exists an MSIS adversary $\mathcal{B}$ such that:*

$$\text{Adv}_{\text{TL-DSA}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \frac{q_H + q_S + 1}{3^{243}} + \text{Adv}_{k, l+1, 1}^{\text{MSIS}}(\mathcal{B}),$$

where the MSIS instance has parameters $(k, l+1, \beta = 1)$ over R_3 with $n = 256$. The running time of $\mathcal{B}$ is that of $\mathcal{A}$ plus $O(q_H \cdot k \cdot l \cdot n^2)$ for polynomial arithmetic.

5.3 Proof of Theorem 5.2

The proof follows the standard Fiat–Shamir signature security framework, adapted to exploit the ternary non-abort property.

Game 0. This is the real EUF-CMA experiment. $\mathcal{A}$ receives a public key $(\rho, \mathbf{t})$ where $\mathbf{t} = \mathbf{A} \cdot \mathbf{s}_1$, has access to signing oracle $\text{Sign}(sk, \cdot)$ and random oracle $\mathcal{H}(\cdot)$, and outputs a forgery attempt $(\mu^*, \sigma^* = (\mathbf{z}^*, \tilde{c}^*))$.

Game 1. We replace the random oracle $\mathcal{H}$ with a lazily-sampled table, answering each fresh query with a uniformly random element of $\{-1, 0, +1\}^{243}$. This is a purely syntactic change:

$$\Pr[\text{Game 1 succeeds}] = \Pr[\text{Game 0 succeeds}].$$

Game 2. We simulate the signing oracle without using $\mathbf{s}_1$. By Theorem 4.3, signatures can be perfectly simulated:

For each signing query on message μ_i :

1. Sample $\mathbf{z}_i \xleftarrow{\$} R_3^l$.
2. Compute $\bar{\mu}_i = \mathcal{H}(\text{trits}(\mathbf{t}) \parallel \mu_i)$.
3. Sample $\tilde{c}_i \xleftarrow{\$} \{-1, 0, +1\}^{243}$.
4. Compute $c_i = \text{SampleChallenge}(\tilde{c}_i)$.
5. Compute $\mathbf{w}_i = \mathbf{A} \cdot \mathbf{z}_i - c_i \cdot \mathbf{t}$.
6. Program $\mathcal{H}(\bar{\mu}_i \parallel \text{trits}(\mathbf{w}_i)) := \tilde{c}_i$.

Programming fails only if $(\bar{\mu}_i \parallel \text{trits}(\mathbf{w}_i))$ was previously queried to $\mathcal{H}$, which happens with probability at most $(q_H + q_S)/3^{243}$ per query (since $\mathbf{w}_i$ depends on the freshly sampled $\mathbf{z}_i$). By a union bound over q_S signing queries:

$$|\Pr[\text{Game 2}] - \Pr[\text{Game 1}]| \leq \frac{q_S(q_H + q_S)}{3^{243}}.$$

Since 3^{243} is astronomically large ($\approx 2^{385}$), this term is negligible for any polynomial q_S, q_H .

Game 2 analysis: extracting an MSIS solution.

In Game 2, the signing oracle does not use $\mathbf{s}_1$. Therefore, $\mathcal{A}$'s view depends on $\mathbf{s}_1$ only through the public key $\mathbf{t} = \mathbf{A} \cdot \mathbf{s}_1$. Suppose $\mathcal{A}$ outputs a valid forgery $(\mu^*, \mathbf{z}^*, \tilde{c}^*)$ where $\text{Verify}(pk, \mu^*, (\mathbf{z}^*, \tilde{c}^*)) = 1$. Then:

$$\mathbf{A} \cdot \mathbf{z}^* - c^* \cdot \mathbf{t} = \mathbf{w}^*, \tag{1}$$

$$\mathcal{H}(\bar{\mu}^* \parallel \text{trits}(\mathbf{w}^*)) = \tilde{c}^*, \tag{2}$$

where $c^* = \text{SampleChallenge}(\tilde{c}^*)$.

Since $\mathbf{t} = \mathbf{A} \cdot \mathbf{s}_1$, equation (1) gives:

$$\mathbf{A} \cdot \mathbf{z}^* - c^* \cdot \mathbf{A} \cdot \mathbf{s}_1 = \mathbf{w}^* \implies \mathbf{A} \cdot (\mathbf{z}^* - c^* \cdot \mathbf{s}_1) = \mathbf{w}^*.$$

Now we apply the **forking lemma**. We rewind $\mathcal{A}$ to the point where $\tilde{c}^*$ was programmed and provide a different random oracle response $\tilde{c}^{**} \neq \tilde{c}^*$. With probability at least $1/(q_H + q_S + 1)$ (by the general forking lemma), $\mathcal{A}$ produces a second valid forgery $(\mu^*, \mathbf{z}^{**}, \tilde{c}^{**})$ on the same message with the same $\mathbf{w}^*$ but different challenge $c^{**} = \text{SampleChallenge}(\tilde{c}^{**})$.

From the two forgeries:

$$\begin{aligned}\mathbf{A} \cdot \mathbf{z}^* - c^* \cdot \mathbf{t} &= \mathbf{w}^*, \\ \mathbf{A} \cdot \mathbf{z}^{**} - c^{**} \cdot \mathbf{t} &= \mathbf{w}^*.\end{aligned}$$

Subtracting:

$$\mathbf{A} \cdot (\mathbf{z}^* - \mathbf{z}^{**}) = (c^* - c^{**}) \cdot \mathbf{t} = (c^* - c^{**}) \cdot \mathbf{A} \cdot \mathbf{s}_1.$$

Therefore:

$$\mathbf{A} \cdot [(\mathbf{z}^* - \mathbf{z}^{**}) - (c^* - c^{**}) \cdot \mathbf{s}_1] = \mathbf{0}.$$

Define the vector $\mathbf{v} = \begin{pmatrix} \mathbf{z}^* - \mathbf{z}^{**} \\ -(c^* - c^{**}) \end{pmatrix} \in R_3^{l+1}$, and extend the matrix as $\mathbf{A}' = [\mathbf{A} \mid \mathbf{t}] \in R_3^{k \times (l+1)}$. Then:

$$\mathbf{A}' \cdot \mathbf{v} = \mathbf{0}, \quad \|\mathbf{v}\|_\infty \leq 1, \quad \mathbf{v} \neq \mathbf{0} \text{ (since } c^* \neq c^{**}\text{)}.$$

This is a valid solution to the $\text{MSIS}_{k,l+1,1}$ problem on the matrix $\mathbf{A}'$.

Combining the bounds. We construct $\mathcal{B}$ as follows: $\mathcal{B}$ receives an MSIS challenge $\mathbf{A}'$, parses it as $[\mathbf{A} \mid \mathbf{t}]$, runs $\mathcal{A}$ in Game 2 using $(\rho, \mathbf{t})$ as the public key (where ρ is set to reproduce $\mathbf{A}$), and applies the forking lemma to extract $\mathbf{v}$.

$$\text{Adv}_{\text{TL-DSA}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \frac{q_S(q_H + q_S)}{3^{243}} + (q_H + q_S + 1) \cdot \text{Adv}_{k,l+1,1}^{\text{MSIS}}(\mathcal{B}).$$

Since $3^{243} \approx 2^{385}$, the first term is negligible for any polynomial number of queries. $\square$

6 Module-SIS to Module-LWE Duality

The hardness of MSIS over R_3 can be related to the hardness of MLWE over R_3 via the standard lattice duality results.

Theorem 6.1 (SIS–LWE duality, adapted from [5, Theorem 4.1]). *For the ring $R_3 = \mathbb{Z}_3[X]/(X^{256} + 1)$, solving $\text{MSIS}_{k,l,\beta}$ with $\beta = 1$ is at least as hard as solving $\text{MLWE}_{k,\chi}$ where χ is any distribution over R_3 . Formally, for any MSIS solver $\mathcal{B}$, there exists an MLWE distinguisher $\mathcal{D}$ such that:*

$$\text{Adv}_{k,l,1}^{\text{MSIS}}(\mathcal{B}) \leq \text{Adv}_{k,\chi}^{\text{MLWE}}(\mathcal{D}) + \frac{1}{3^{nk}}.$$

Proof sketch. The proof follows the standard approach. Given an MLWE instance $(\mathbf{A}, \mathbf{b})$ where either $\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$ or $\mathbf{b} \xleftarrow{\$} R_3^k$:

If $\mathbf{b}$ is uniform, then $[\mathbf{A} \mid \mathbf{b}]$ is a uniform matrix in $R_3^{k \times (k+1)}$, and any MSIS solution on this matrix would solve a uniformly random MSIS instance.

If $\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$, then $[\mathbf{A} \mid \mathbf{b}]$ has the hidden short vector $(\mathbf{s}^\top \mid -1)^\top$ satisfying $[\mathbf{A} \mid \mathbf{b}] \cdot (\mathbf{s}^\top \mid -1)^\top = \mathbf{e}$ (which is short). An MSIS solver that finds *any* short vector in the kernel would either find this planted solution or an independent one—either way, the existence of a short kernel vector distinguishes the two cases.

Over R_3 , the key observation is that $\beta = 1$ and all elements already satisfy $\|\cdot\|_\infty \leq 1$, so the “shortness” condition is automatically satisfied. The hardness therefore reduces to finding *any* nontrivial kernel element of a random matrix over R_3 , which is the core MLWE/MSIS problem in the ternary setting. $\square$

Corollary 6.2. *The EUF-CMA security of TL-DSA reduces to the hardness of Module-LWE over R_3 :*

$$\text{Adv}_{\text{TL-DSA}}^{\text{EUF-CMA}}(\mathcal{A}) \leq \frac{q_S(q_H + q_S)}{3^{243}} + (q_H + q_S + 1) \cdot \left(\text{Adv}_{k,\chi}^{\text{MLWE}}(\mathcal{D}) + \frac{1}{3^{nk}} \right).$$

7 Concrete Security Estimates

7.1 Lattice Dimension and Hardness

The concrete security of TL-DSA depends on the hardness of finding short vectors in Module-LWE/SIS lattices over R_3 .

For a Module-LWE instance with matrix $\mathbf{A} \in R_3^{k \times l}$ and polynomial degree $n = 256$, the underlying lattice has rank $d = (k + l) \cdot n$ over $\mathbb{Z}_3$. The Hermite factor δ required to solve this lattice problem using BKZ with block size b satisfies:

$$\delta^d \approx \frac{3^{nk/d}}{\sqrt{2\pi e}},$$

and the best known classical algorithms achieve $\delta \approx \left(\frac{\pi b}{2\pi e} \cdot (\pi b)^{1/b}\right)^{1/(2(b-1))}$.

Table 2: Concrete security estimates for TL-DSA parameter sets.

Variant	k	l	Lattice rank d	Estimated BKZ- b	Classical bits	Quantum bits
TL-DSA-44	4	4	2048	≥ 356	≥ 128	≥ 116
TL-DSA-65	6	5	2816	≥ 538	≥ 192	≥ 174
TL-DSA-87	8	7	3840	≥ 720	≥ 256	≥ 232

The estimates use the Core-SVP methodology: classical security is $0.292b$ bits (sieving) and quantum security is $0.265b$ bits (Grover-accelerated sieving), consistent with the methodology used in the ML-DSA specification.

7.2 Hash Function Security

The ternary sponge has capacity 486 trits ($= 3^{486} \approx 2^{770}$ states). Collision resistance is bounded by the birthday attack at $\sqrt{3^{486}} \approx 2^{385}$ operations. Preimage resistance is $3^{486} \approx 2^{770}$ operations. Both are superexponential in the security parameter for all TL-DSA variants.

The challenge space has $\binom{256}{\tau} \cdot 2^\tau$ elements (choosing τ positions from 256 and assigning each ± 1):

Table 3: Challenge space sizes.

Variant	τ	$\log_2 \left(\binom{256}{\tau} \cdot 2^\tau \right)$	Collision security
TL-DSA-44	39	≈ 178	≥ 89 bits
TL-DSA-65	49	≈ 213	≥ 106 bits
TL-DSA-87	60	≈ 248	≥ 124 bits

The challenge collision security is the binding constraint for TL-DSA-44 and TL-DSA-65. For TL-DSA-87, the lattice dimension is the binding constraint.

7.3 Sizes

Byte sizes computed as $\lceil \text{trits} \times 2/8 \rceil$ (2 bits per trit in packed encoding, matching the `to_bytes` implementation in `ternary_lattice.rs` line 217).

Table 4: Key and signature sizes (in trits and bytes).

Variant	$ pk $ (trits)	$ pk $ (bytes)	$ sk $ (trits)	$ sk $ (bytes)	$ \sigma $ (trits)	$ \sigma $ (bytes)
TL-DSA-44	1267	317	3315	829	1267	317
TL-DSA-65	1779	445	4595	1149	1523	381
TL-DSA-87	2291	573	6131	1533	2035	509

8 Discussion

8.1 Comparison to ML-DSA

TL-DSA and ML-DSA share the same high-level structure (Fiat–Shamir with Aborts over Module-LWE lattices) but differ in three significant ways:

1. **Ring modulus.** ML-DSA uses $q = 8380417$ with NTT-friendly factorization. TL-DSA uses $q = 3$, which precludes NTT but enables the non-abort property. Ring multiplication is $O(n^2)$ schoolbook rather than $O(n \log n)$ NTT.
2. **Rejection sampling.** ML-DSA requires ~ 4 – 7 signing attempts on average. TL-DSA requires exactly one (Theorem 4.1), yielding deterministic signing with perfect zero-knowledge.
3. **Key and signature sizes.** TL-DSA produces significantly smaller keys and signatures because each coefficient is a trit (2 bits packed) rather than a $\lceil \log_2 q \rceil = 23$ -bit integer. However, the security per trit of lattice dimension is lower because the lattice is over a smaller ring.

8.2 Security Considerations Specific to $q = 3$

The small modulus $q = 3$ has implications that require careful analysis:

Algebraic attacks. Over $\mathbb{Z}_3[X]/(X^{256} + 1)$, the polynomial $X^{256} + 1$ factors nontrivially modulo 3 (since $X^{256} + 1 = (X^{128})^2 + 1$ and $-1 \equiv 2 \pmod{3}$). The factorization structure could potentially be exploited by algebraic attacks that do not apply to NTT-friendly primes. The security estimates in Table 2 assume generic lattice attacks; algebraic speedups are an open research question.

Entropic hardness. When $q = 3$, the secret $\mathbf{s}_1$ and the uniform distribution have the same support ($\{-1, 0, +1\}$). The MLWE problem reduces to distinguishing $\mathbf{A} \cdot \mathbf{s}$ from uniform, which requires exploiting the algebraic structure of $\mathbf{A}$ rather than the “smallness” of $\mathbf{s}$. The module rank (k, l) must therefore be sufficiently large to compensate for the lack of a noise-to-modulus ratio.

8.3 Open Problems

1. **Tight reduction.** The forking lemma introduces a q_H -factor loss in the reduction. Achieving a tight reduction (without the forking lemma) via the lossy identification paradigm is an open question for TL-DSA.
2. **Algebraic factorization analysis.** A complete analysis of the factorization of $X^{256} + 1$ over $\mathbb{Z}_3$ and its impact on lattice attacks would strengthen the concrete security estimates.
3. **Formal security of the ternary sponge.** The proof assumes the ternary sponge is a random oracle. A dedicated cryptanalysis establishing the sponge’s indistinguishability from a random oracle would close this gap (see companion report: Ternary Sponge Cryptanalysis, Task 2.5).

9 Conclusion

We have established the EUF-CMA security of TL-DSA under the Module-LWE hardness assumption over the ternary polynomial ring $R_3 = \mathbb{Z}_3[X]/(X^{256} + 1)$, in the random oracle model. The security reduction proceeds via a standard Fiat–Shamir argument with forking, with the key innovation that the ternary non-abort property (Theorem 4.1) eliminates rejection sampling entirely, yielding deterministic signing with perfect zero-knowledge simulation. Concrete security estimates confirm that the three parameter sets (TL-DSA-44/65/87) achieve their targeted 128/192/256-bit classical security levels under standard lattice attack cost models.

References

- [1] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé. CRYSTALS–Dilithium: A Lattice-Based Digital Signature Scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, 2018.
- [2] V. Lyubashevsky. Lattice Signatures without Trapdoors. In *EUROCRYPT 2012*, LNCS 7237, pages 738–755. Springer, 2012.
- [3] A. Fiat and A. Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *CRYPTO 1986*, LNCS 263, pages 186–194. Springer, 1986.
- [4] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA). August 2024.
- [5] C. Peikert. A Decade of Lattice Cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4):283–424, 2016.
- [6] A. Langlois and D. Stehlé. Worst-Case to Average-Case Reductions for Module Lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.
- [7] E. Kiltz, V. Lyubashevsky, and C. Schaffner. A Concrete Treatment of Fiat-Shamir Signatures in the Quantum Random Oracle Model. In *EUROCRYPT 2018*, LNCS 10822, pages 552–586. Springer, 2018.
- [8] D. Pointcheval and J. Stern. Security Arguments for Digital Signatures and Blind Signatures. *Journal of Cryptology*, 13(3):361–396, 2000.

A Implementation Correspondence

The following table maps each algorithmic step to the production Rust implementation in the PlenumNET repository.

B Test Vector Correspondence

The following unit tests in `tl_dsa.rs` validate the implementation against the formal specification:

- `test_keygen_deterministic` (line 422): Verifies that KeyGen is deterministic given the same seed.
- `test_sign_verify_44` (line 438): End-to-end correctness for TL-DSA-44.
- `test_sign_verify_wrong_message` (line 449): Forgery detection (wrong message).

Table 5: Algorithm-to-code correspondence.

Algorithm Step	Source File	Lines
R_3 ring definition	<code>ternary_lattice.rs</code>	58–62, 95–99
<code>mod3</code> reduction	<code>ternary_lattice.rs</code>	67–75
Ring multiplication (schoolbook)	<code>ternary_lattice.rs</code>	185–214
CBD_η sampling	<code>ternary_lattice.rs</code>	423–449
Uniform ternary sampling	<code>ternary_lattice.rs</code>	401–421
Matrix expansion	<code>ternary_lattice.rs</code>	452–461
Parameter sets	<code>tl_dsa.rs</code>	73–103
KeyGen	<code>tl_dsa.rs</code>	198–231
Sign (with abort loop)	<code>tl_dsa.rs</code>	254–308
Verify	<code>tl_dsa.rs</code>	310–348
SampleChallenge	<code>tl_dsa.rs</code>	157–183
Ternary sponge hash	<code>sponge.rs</code>	46–69 (permutation)
ℓ_∞ norm	<code>ternary_lattice.rs</code>	141–143

- `test_sign_verify_wrong_key` (line 462): Forgery detection (wrong key).
- `test_sign_deterministic` (line 478): Verifies deterministic signing (Corollary 4.2).
- `test_sample_challenge` (line 514): Verifies challenge weight bound $\text{wt}(c) \leq \tau$.

All tests pass as of the March 2026 audit.